

Word 数据结构竞品分析

背景：

腾讯文档 Word 页面内容过多时，atext 数据转换慢，首屏可见时间很晚，需要数据分片，加快首屏可见时间。

早期其他同事分析谷歌文档数据：<http://km.oa.com/articles/show/415990>

一、腾讯文档

Word 当前文本数据的结构主要如下：

- 1) 数据段开始为正文文本数据段，以`\u000f\u0003`为正文文本段结束标记；
- 2) 文本数据段中可任意位置插入类似`\*`或`\u0008`等表示特殊占位符的文本，这些特殊占位符和对应的属性关联表示特殊的数据内容。（例如图片，代码块，第三方内容等）
- 3) 正文文本数据段后跟随的是子流数据段，每个子流数据段都以`\u0002`和`\u0003`分别标记为子流开始符和子流结束符；

```
1  正文*内容\u0008 其他文本\n
2  \u000f\u0003
3  \u0002 页眉\u0003
4  \u0002 子流\u0003
```

服务器拉取的数据格式：

- 1 服务器下来的数据为 Changeset + apool 的组合；
- 2 1.其中 Changeset 包含 text 文本段内容和 attribs 中的字属性和段属性等；
- 3 2.apool 为属性池 map；

示例：

text："正文内容 123\n\u000f\u0003\u0002 页眉\u0003..."

attribs："*0+7+1*1*2+1*3+1*4*5+1*4*5+1*6*7+1*0*8+2*9*a*0*8+1..."

二、谷歌文档（bingolin）

方案描述：Html 中直出数据分片

Google 直出的 Html 中直出了完整的文档数据分片，其中每个数据片由一个<scrips>标签承载，每个数据分片为一个对象 model_chunk，大致结构如下所示：

```

DOCS_modelChunkParseStart = new Date().getTime();
</script>
<script type="text/javascript" nonce="1mZop907mU/3461QYHEQ3Q">
DOCS_modelChunk = [{
  "ty": "is",
  "ibi": 16911,
  "s": "\"Never you mind what it is,\" the guard retorted. \"What are you?\"\\n\"Is tha
}, {
  "ty": "as",
  "st": "paragraph",
  "si": 16975,
  "ei": 16975,
  "sm": {
    "ps_klt_i": true,
    "ps_awao_i": true,
    "ps_ls": 1.3,
    "ps_sm_i": true,
    "ps_sm_i": 5.13
  }
}];

```

```

1 DOCS_modelChunk = [{
2     type: 'text', // 主文档的文本数据流
3     index: 1, // 本次数据分片的文本索引位置，第一片 1，第一片长度为 100 则第二
      片的索引起始位置为 101
4     text: "第一段：123456\\n 第二段：123456\\n 第三段：
      123456\\n*\\n\\u0010\\u0012\\u001c1\\n\\u001c2\\n\\u0012\\u001c3\\n\\u001c4\\n\\u0011",
5 }, {
6     type: 'text', // 页眉页脚的数据流，由\\u0003 划分
7     index: xx // 子流的起始位置
8     text: "\\u0003 这是一段页眉的文字\\n\\u0003 这是一段页脚的文字\\n",
9 }, {
10    type: 'attrib',
11    name: 'paragraph',
12    sldx: 11,
13    eldx: 11,
14    attribs: {...}
15 }, {
16    type: 'attrib',
17    name: 'paragraph',
18    sldx: 22,
19    eldx: 22,
20    attribs: {...} // 段属性 map
21 }, ...,
22 {
23     type: 'attrib',
24     name: 'text',
25     sldx: 0,
26     eldx: 23,
27     attribs: {...} // 字属性 map
28 }];

```

数据分片的说明：

- 1) 每个分片对象是个数组，数组中每个对象可为文本数据，段属性数据，字属性数据等具体的数据对象；
- 2) 每个分片第一个数据对象为正文的文本数据段内容；
- 3) 文章的第一个数据分片对象中，第一个正文文本数据段对象后会接页眉和页脚的文本数据段对象；
- 4) 段属性数据段与字属性数据段是不相关的，且各类属性的数据段序列是连续的（即上一个数据段的区间结束位置与下一个数据段的起始位置可无缝衔接）；

初探数据分片规则：

- 1) 第一个数据分片携带页眉的数据，即使页眉数据很大也会携带完整的页眉数据；
- 2) 一个完整的大表格进行数据分片时总是在表格某一行结束位置进行拆分；
- 3) 每个数据片的大小似乎主要是确保文本内容的字符长度约为略大于 4000 字符的某个值；

优点：

- 1) 完整的数据在首屏中直出，html 加载完后可动态增量加载分片数据，不会丢失分片数据；

缺点：

- 1) 如果文档内容中数据量过大，导致直出的 html 加载时间较长；

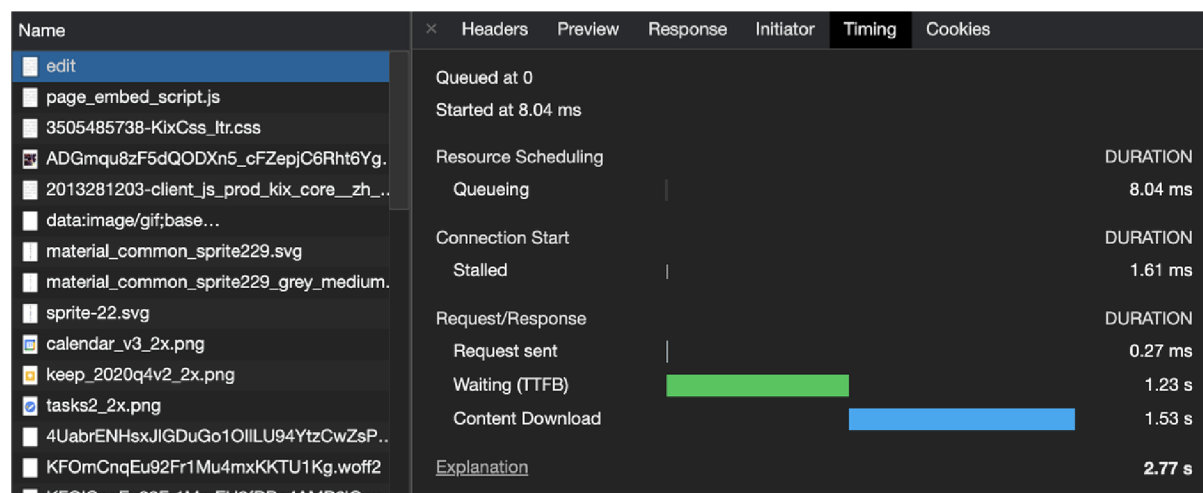


图 1.html 加载 timing（约 20w+文本的文档）

三、金山（bingolin）

方案描述：cgi 拉取首片数据 + ws 拉取增量分片数据

金山文档网页加载过程中，html 中没有进行 SSR 支持，html 中较前置的位置发起 cgi 拉取首片数

据进行渲染，随后通过已经完成建连的 websocket 增量拉取分片数据；

拉取的首片数据分片为一个 json，其中结构和数组字段大致如下：

```

1  {
2    concernRanges: [{ // 数据分片的文本 range 信息
3      type: 1,
4      range: {pos: 0, len: 4341}
5    }],
6    textstream: [{ // 具体的数据信息，可能有多个不同区间的数据
7      pos: 0,
8      length: 4341,
9      textRange: [{ // 文本数据信息
10        pos: 0,
11        length: 4096,
12        text: '双城记第一部 死人复活 第 01 章 时代那是最美好的时代...',
13      }, {
14        pos: 4096,
15        length: 245,
16        text: ' Woodman and that Farmer, though they...'
17      }],
18      chpx: [{ // 字属性区间信息
19        pos: 0,
20        len: 3,
21        propBag: {...} // 字属性包数据
22      }, {
23        pos: 3,
24        len: 1,
25        propBag: {...} // 字属性包数据
26      }, ...],
27      papx: [{ // 段属性区间信息
28        pos: 0,
29        len: 4,
30        propBag: {...} // 段属性包数据
31      }, {
32        pos: 4,
33        len: 17,
34        propBag: {...} // 段属性包数据
35      }, ...],
36    }],
37  }

```

初探数据分片规则：

- 1) 拉取的首片数据中页眉页脚的数据是完整的，且进行了分段；
- 2) 上述一个完整的数据片中，textstream.textRange 里划分的每个文本段长度约为略大于4000 字符的值(4000 指不固定)；

PS：上述的数据分片中，实际上不仅是包含有文本和属性数据，还包括了部分排版信息的数据，例如每一页的大小和位置，页中每行的大小位置和对应的文本数据区间。故金山文档的页面打开后首屏渲染可能是首片数据+首片数据对应的排版信息综合快速生成的。

优点：

- 1) 数据分片，首屏渲染只请求首屏数据，首屏排版和渲染可较快；
- 2) 数据增量拉取，减少首屏拉取数据量耗时，ws 通道稳定性和加载速度较好；

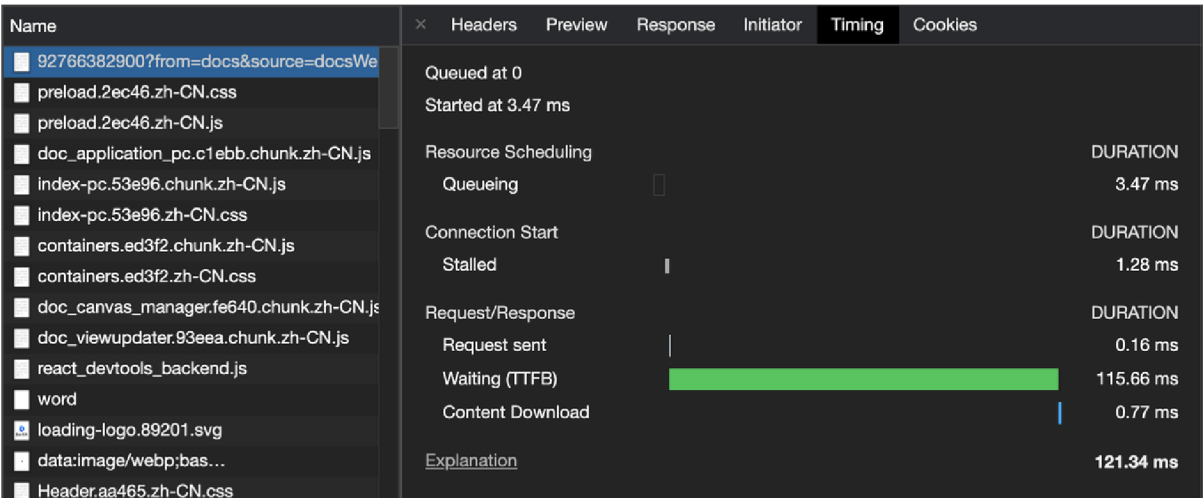


图 1.html 加载 timing (约 20w 文本的文档)

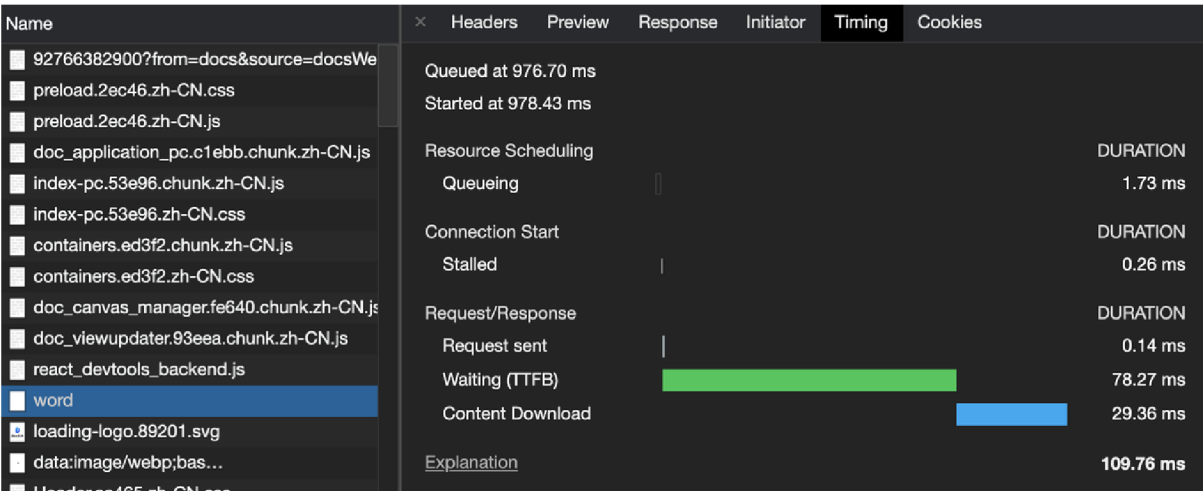


图 2.首屏数据拉取 cgi 加载 timing (约 20w 文本的文档)

缺点：

- 1) 数据分片增量加载，需要处理数据分片的连续性问题；（ PS：还要考虑上本地离线数据应用，增量加载过程中的协同等场景 ）

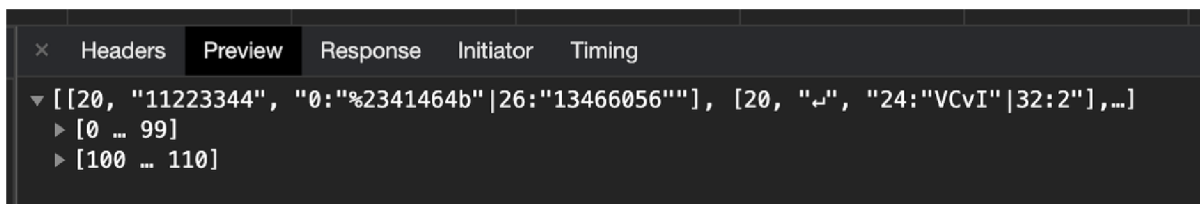
四、石墨 (joriswang)

方案描述：html 加载时通过 CGI 拉取全量数据，不支持 SSR 和分片

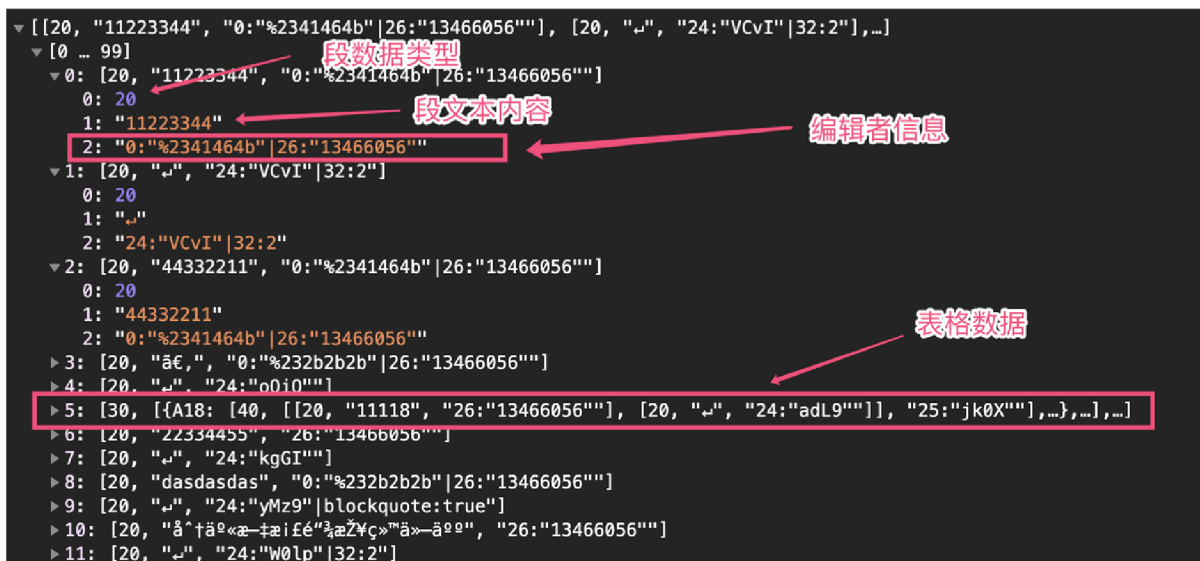
石墨文档打开过程不支持 SSR，html 中不包含数据，全文数据通过 CGI 请求单次全量拉取到本地，然后全量排版渲染，因此大文档（使用 5W 字多格式文档）首屏可见非常慢，过程中 CPU 占用接近 100%。

数据结构：

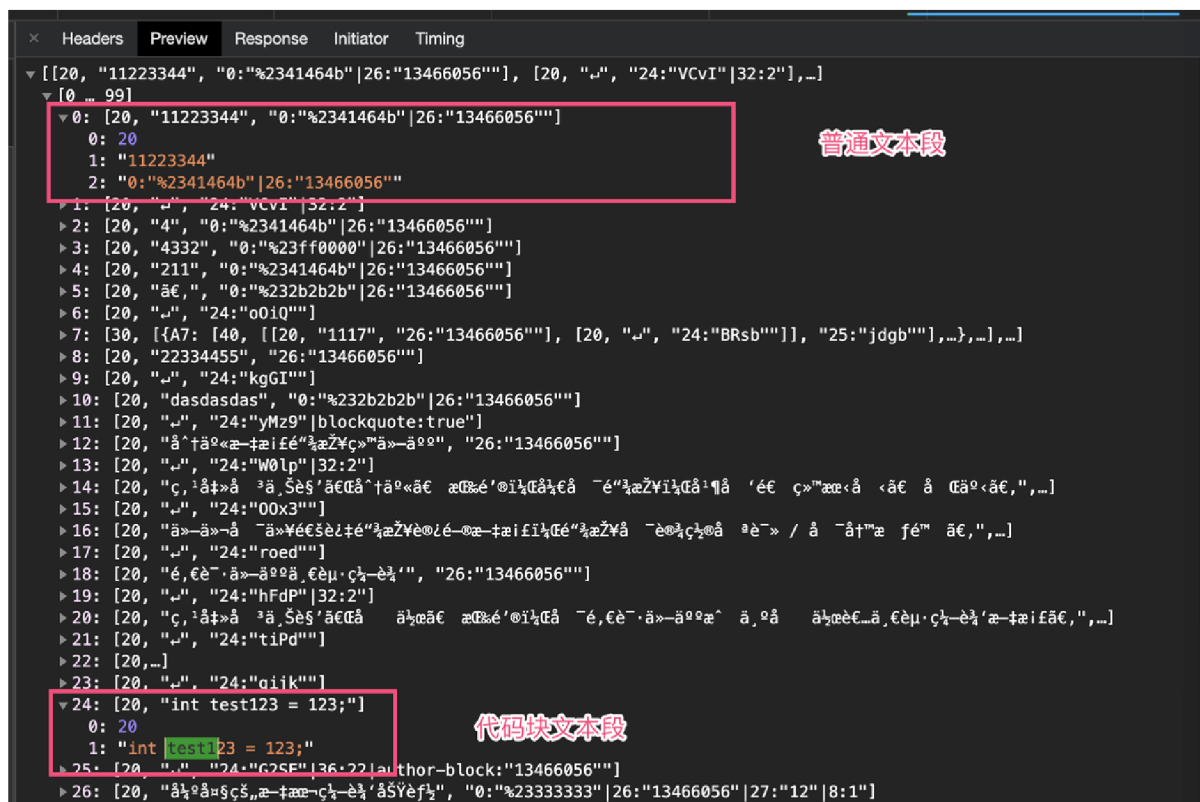
全文数据是结构化的 json 格式，可读性较好。从结构上看，对分片支持良好，是以段为单位，若干个段可以合并成一片，见下图：



段基本结构如下图：



表格数据是包含在正文数据以内，作为一个段，用类型区分，基本数据结构如下图：



数据结构跟腾讯文档一样，还是原始的 **text+attrs** 线性结构，可读性较差，不支持分片。

六、总结

1、竞品总结

	SSR	数据分片	首屏数据来源	分片数据来源
腾讯文档	是	否	CGI 拉取	无
谷歌文档	否	是	跟随文档直出	跟随文档直出
金山文档	否	是	CGI 拉取	Websocket 增量拉取
石墨	否	否	CGI 拉取	无
Lark	桌面端：是 移动端：否	否	桌面端：随文档直出 移动端：CGI 拉取	无

2、文档可借鉴点

- 1) 首屏只拉取首片数据进行渲染 (cgi 拉取或 html 中直出首片数据两种方式可再探讨);
- 2) 数据分片的结构可以采用金山类似的结构, 与现有数据层类似可快速进行转换;

3、待处理的问题点

- 1) 现有 cs 格式的数据转换为直出数据格式的方案; (重点)
方案一: 直接用 easysync 接口迭代 cs 进行解析处理转换为数据分片对象 cs 子序列; (放弃)
方案二: 数据层 textstream 那套逻辑后台跑起, 直接序列化数据层对象到前端反序列化, 小文档数据跟随 SSR 数据直出, 大文档数据跟随 opendoc 直出;
- 2) 转换后的分片数据格式在前后端传输中的序列化方式; (p1)

4、文档数据结构

不升级数据结构的情况下, 直出数据直接使用数据层导出的 json 格式。

[隐私保护](#)
[权利保护](#)
[闽ICP备16038489号-5](#)
 © 2014 - 2020 Json.cn All right reserved.
 邮箱: 2340592@qq.com

 保留转义 **阿里云双十一最低价** **腾讯云双11最划算**